

Informatique Fondamentale
CNAM-IM-1A

jeanmarc.robert83

March 2025

Chapitre 1

Introduction à la qualité numérique

1.1 Introduction

Le besoin d'effectuer des calculs numériques soulève la question de la représentation des nombres. Pour les entiers, c'est simple, la représentation binaire s'est imposée, et la limite des nombres entiers représentables est conditionnée par le nombre de bits ou chiffres binaires dont on dispose. Typiquement, un registre peut comporter 32 ou 64 bits dans les processeurs modernes courants. La question du signe est réglée par l'utilisation d'un bit de signe, ou le plus souvent avec une représentation en complément à deux (on représente $-x$ par $2^n - x$).

Mais si l'on souhaite représenter des nombres non entiers, la question se complique. Assez naturellement, on pense à la représentation en virgule fixe. Les nombres non entiers sont représentés comme des entiers, affectés par un décalage, c'est à dire une division par une puissance de la base. L'avantage de cette représentation est que les opérations s'effectuent comme sur des entiers. L'inconvénient saute aux yeux immédiatement : la dynamique du système est très limitée, et l'écart entre deux nombres à virgule fixe consécutifs est constant. Les erreurs d'arrondi peuvent prendre des valeurs catastrophiques très rapidement.

La représentation en virgule flottante tente de répondre à ces questions. Toutefois, elle ne sont pas parfaites et donnent lieu aussi à des erreurs numériques dans les calculs. Pire, ces erreurs peuvent se cumuler et aboutir à des résultats parfois fantaisistes. Et les conséquences de ces erreurs ont parfois donné lieu à des événements historiques surprenants : krach boursiers, ratage d'opérations militaires, fusées qui s'écrasent...

Le but de cette petite introduction est de permettre de maîtriser la représentation des nombres flottants, les algorithmes des opérations principales et de saisir les principaux enjeux autour de la qualité numérique.

1.2 Représentation flottante

Les processeurs implantent (normalement) la norme IEEE754, qui définit la représentation des nombres flottants.

1.2.1 Définition de la représentation flottante

La représentation est décrite dans Goossens [1] par exemple :

Un nombre est représenté par le triplet (s, e, m) . ces valeurs respectent les conditions suivantes :

$$\begin{aligned} s &\in \{0, 1\} \\ 1 &\leq e < 2b + 1 \\ 2^{p-1} &\leq M < 2^p \text{ pour un flottant normalisé} \end{aligned}$$

Ceci signifie que pour les flottants normalisés, on peut aussi écrire :

$$M = 2^p + m, \text{ avec } 0 \leq m < 2^{p-1}$$

Un nombre flottant normalisé x s'écrit alors

$$x = (-1)^s \times 2^{e-b} \times \left(1 + \frac{m}{2^{p-1}}\right)$$

avec p la précision, c'est le nombre de bits de M , et le biais $b = 2^{\ell_e-1} - 1$ si ℓ_e est le nombre de bits de l'exposant.

La table suivante donne les paramètres b et p :

Type taille (bits)	simple 32	double 64	half 16
b	127	1023	15
p^*	23+1	52+1	10+1

* : nombre de bits de la mantisse comprend le bit implicite à 1 pour les flottants normalisés, c'est le bit de poids fort qui est toujours à 1.

Dans la suite, on notera $\mathbb{F}$ l'ensemble des nombres flottants dans l'une ou l'autre précision.

1.2.2 Modes d'arrondis

La norme IEEE754 définit aussi la façon d'arrondir les résultats :

- exige l'arrondi correct
- les différents modes : au plus près, vers le haut, vers le bas, vers zéro...

Le mode au plus près nécessite de définir ce que l'on fait en cas de point milieu. Deux possibilités :

1. vers le flottant le plus proche avec un chiffre de mantisse de poids faible pair. C'est souvent le mode par défaut.
2. de choisir le nombre flottant le plus proche en s'éloignant de zéro, soit la mantisse la plus importante en valeur absolue.

1.2.3 Des cas particuliers et des exceptions !

Quelques cas particuliers définis par la norme :

- $e = 0$ et $m = 0$: dans ce cas, le nombre est ± 0 en fonction de s
- $e = 2b + 1$ et $m = 0$: le flottant est alors $\pm\infty$
- $e = 2b + 1$ et $m \neq 0$: le flottant est alors NaN , pour *Not a Number*

- $e = 0$ et $m \neq 0$: le nombre flottant est dit *dénormalisé*. Il s'exprime alors de la façon suivante :

$$x = (-1)^s \times 2^{-b+1} \times (0 + \frac{m}{2^p})$$

On ne mentionne pas ici toutes les propriétés de flottants, mais il faut savoir que :

- tous les nombres réels ne sont pas représentables. C'est une évidence : $\#\mathbb{R} = \infty$ alors que $\#\mathbb{F} < 2^w$, avec $w = 16, 32, 64$ respectivement dans le cas de *half*, simple ou double respectivement.
- L'écart entre deux nombres flottants consécutifs n'est pas constant, contrairement aux représentations en virgule fixe.
- les opérations arithmétiques ne sont pas associatives. Il faut en tenir compte dans les transformations de programme que l'on envisage, même si cela peut arriver avec des jeux de données particuliers. De même, il arrive fréquemment que pour $x \in \mathbb{F}$, on ait $1/x \notin \mathbb{F}$, par exemple $x = 10.0$ en *half*, simple ou double précision.
- il existe d'autres formats pour des précisions plus grandes, voire des bibliothèques multiprécisions (GMP, MPFR...)
- la norme définit aussi des systèmes décimaux. La définition d'un nombre flottant dans un tel système est :

$$x = (-1)^s \times 10^{e-b} \times (1 + \frac{m}{10^{p-1}})$$

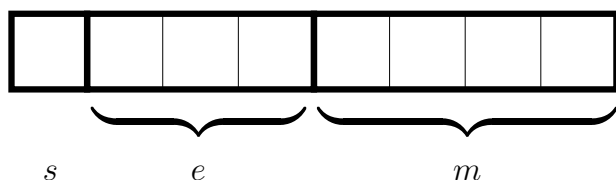
Dans ce cas, $p - 1$ est le nombre de chiffres décimaux, soit $0 \leq m_i < 10$.

L'ensemble des nombres flottants est donc un sous-ensemble très particulier des nombres rationnels, lui-même sous-ensemble des nombres réels. Mais autant les nombres rationnels présentent des propriétés permettant des transformations d'écritures sans risque et donc des preuves aisées sur des fonctions ou expressions, ce n'est pas le cas des flottants. Ainsi, l'ensemble des nombres rationnels munis de l'addition et de la multiplication $(\mathbb{Q}, +, 0, \times, 1)$ est un sous-corps des réels. L'ensemble des flottants $(\mathbb{F}, +, 0)$ n'est même pas un groupe ! De plus :

- l'addition et la multiplication ne sont pas associatives ;
- la multiplication n'est pas distributive pour l'addition (la somme des produits n'est pas toujours égale au produit de la somme) ;
- L'inverse d'un nombre pour la multiplication n'existe pas toujours, i.e., on n'a pas toujours la garantie de l'existence de $1/x \in \mathbb{F}$ pour tout $x \in \mathbb{F}$

Exercice 1. Format jouet sur 8 bits.

On considère le système suivant :



Pour comprendre comment fonctionne la représentation flottante, voici quelques questions :

1. Donner les bornes minimales et maximales de s, e, m ainsi que la valeur des paramètres b, p .

2. Si on considère les nombres normalisés, quel est le plus petit et le plus grand nombre représentable dans ce système ? Même question pour les dénormalisés.
3. Représenter sur une droite les nombres représentés par ce système. On tracera les nombres normalisés et dénormalisés. Quel constat pouvez-vous faire à partir de ce tracé ?
4. Que représentent les octets suivants avec les paramètres proposés ?

(a)

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

(b)

1	0	1	0	0	0	1	0
---	---	---	---	---	---	---	---

(c)

0	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

(d)

1	0	0	0	1	0	1	0
---	---	---	---	---	---	---	---

(e)

0	0	1	1	0	0	0	0
---	---	---	---	---	---	---	---

5. Quel est l'écart entre deux flottants consécutifs, en fonction des paramètres et de l'exposant ?
6. Inversement, comment sont représentés les nombres de la liste ci-après, en base 10 ?

$\{0.125, 0.875, -6.2, 5.125\}$

Attention, cette question peut poser le problème de l'arrondi. Vous utiliserez le mode d'arrondi par défaut défini dans la norme IEEE754 présenté plus haut.

1.3 Comment calculer avec des flottants ?

1.3.1 Algorithmes

Pour s'appropriier la représentation flottante, rien de tel que de l'implanter soi-même. L'exercice suivant propose de relever ce défi.

Exercice 2. Pour faire le lien entre la représentation flottante et les nombres représentés, on propose d'utiliser une variable de type `union` comme suit :

```
#define S_INT 32
#define F_BIAS 127
#define S_MANTISSA 23
```

```
union nf{
    int i;
    float f;
};
```

Ceci permet de traiter un même mot de 32 bits indifféremment comme un `float` ou un `int` sans risquer des conversions de type intempestives. Par exemple on peut écrire :

```
union nf x;
int n=0x3f800000;
x.i = n;
printf("x.i=0x%x, x.f=%f\n",x.i,x.f);
```

Et obtenir la sortie suivante :

```
x.i=0x3f800000, x.f=1.000000
```

Questions et travail à réaliser :

1. La sortie donnée en exemple est-elle juste ? Procédez aux vérifications nécessaires.
2. Écrivez une fonction qui extrait le signe, l'exposant et la mantisse d'un nombre flottant ayant le prototype suivant :

```
int affsem(unsigned int *s, unsigned int *e, unsigned int *m, unsigned int n)
{
    // à compléter
    return 0;
}
```

N'oubliez pas d'ajouter le bit implicite à la mantisse.

3. Écrivez une fonction qui convertit un entier en flottant, qui exploite le type `union`, ayant le prototype suivant :

```
union nf fromIntToFloat(int n){
    union nf infl;
    // à compléter
    return infl;
}
```

Consigne : il est interdit d'utiliser les conversions de type proposées par le langage, le but est de faire cette conversion manuellement pour bien comprendre comment fonctionne la représentation.

- extraire d'abord de le signe de l'entier
- calculer l'exposant correspondant
- construire la mantisse, éliminer le bit implicite
- enfin, ranger ces trois champs à leur emplacement respectif dans le mot de 32 bits correspondant.

Retrouvez-vous les résultats donnés en exemple ?

Dans les questions suivantes, vous ne traiterez que les nombres flottants normalisés.

4. Après ce travail, vous pouvez implanter votre addition de flottants maison. Vous implanterez cette opération en simple précision, c'est à dire le type `float`. Vous pourrez exploiter le type `union` évoqué plus haut, ainsi que la fonction de la question 2.

Pour ce faire, vous devez :

- écrire l'algorithme d'addition de nombres flottants, en pseudo-code par exemple. La principale difficulté est de gérer l'arrondi correct.
- écrire le code correspondant et vérifier les résultats.

5. Même question pour la multiplication.

1.3.2 Erreur absolue et erreur relative

L'exercice précédent soulève quelques problèmes et donne lieu à quelques définitions. Par exemple, l'erreur relative d'arrondi est bornée pour tout nombre réel. La borne de cette erreur peut être déterminée.

Définition 1. ε -machine (*epsilon-machine*)

On rappelle qu'un nombre flottant s'exprime :

$$x = \pm 1.m_1m_2 \dots m_{p-1} \times 2^e$$

Le premier nombre flottant supérieur (consécutif) à 1 est donc :

$$1,00\dots001 = 1 + 2^{-(p-1)}$$

On définit l'*epsilon-machine* ε :

$$\varepsilon = 0.00\dots001 = 2^{-(p-1)}$$

Définition 2. *Unit in the Last Place* On note *ulp* la quantité

$$ulp = \varepsilon \times 2^{e-b} = 2^{-(p-1)} \times 2^{e-b}$$

C'est le poids du plus petit bit de la mantisse d'un nombre flottant

$$\begin{cases} ulp(x) & \leftarrow x^+ - x \text{ si } x > 0, \\ ulp(x) & \leftarrow x - x^- \text{ si } x < 0. \end{cases}$$

Exercice 3. Algorithme de Malcolm-Gentleman

On considère une représentation flottante en base 2, avec arrondi correct au plus près et sans dépassement de capacité.

L'algorithme de Malcolm-Gentleman est le suivant :

Require: $A \leftarrow 1.0$ et $B \leftarrow 1.0$

Ensure:

while $((A + 1.0) - A) - 1.0 = 0.0$ **do**

$A \leftarrow 2.0 * A$

end while

while $((A + B) - A) - B \neq 0.0$ **do**

$B \leftarrow B + 1.0$

end while

return B

1. Cet algorithme termine-t-il ? Si oui, dans quelles conditions ?
2. Implanter cet algorithme en C, en utilisant pour A et B les types `float` et `double`.
3. Quelles sont les valeurs de A et de B en sortie de algorithme ?
4. Pouvez-vous relier ces valeurs à l' ε machine et à l'*ulp* ?

Cet exercice met en évidence deux phénomènes influant sur la qualité des résultats :

- L'absorption ;
- la *cancellation*, en français, on pourrait dire l'effacement.

On peut maintenant définir l'erreur absolue et l'erreur relative.

Définition 3. Erreur absolue :

soit un nombre **réel** x tel que :

$$x = \pm 1.m_1m_2 \dots m_pm_{p+1} \dots \times 2^E$$

Ce nombre est potentiellement non représentable en précision finie.

On note $\hat{x} = \text{round}(x)$ l'arrondi de x dans un format en précision p .

On appelle erreur absolue la valeur définie de la manière suivante :

$$\text{abserr}(x) = |\hat{x} - x|$$

Définition 4. Erreur relative

soit un nombre **réel** $x \neq 0$ tel que :

$$x = \pm 1.m_1, m_2 \dots m_pm_{p+1} \dots \times 2^E$$

Ce nombre est potentiellement non représentable en précision finie.

On note $\hat{x} = \text{round}(x)$ l'arrondi de x dans un format en précision p .

$$\text{relerr}(x) = \frac{|\hat{x} - x|}{|x|}$$

Exercice 4. Soit deux nombres flottants consécutifs. Pour simplifier, on considérera uniquement des nombres positifs.

1. Écrire ces nombres flottants en fonction de leur mantisse et exposant. Quels cas particuliers peuvent apparaître ?
2. Dans le cas où ces deux nombres ont le même exposant, calculer l'écart entre ces deux nombres flottants. Quel est le lien avec l' ε et l'*ulp* ?
3. Exprimez l'erreur absolue et l'erreur relative d'arrondi d'un nombre réel quelconque situé entre ces deux nombres flottants, en fonction des grandeurs précédentes.
4. Qu'en est-il des bornes sur ces erreurs ?

La conclusion de cet exercice est fondamentale :

Pour tout x dans l'intervalle des nombres flottants normalisés, on a, quel que soit le mode d'arrondi :

$$|x| < 2^{e-b} \text{ et } \text{relerr}(x) < \frac{2^{-p+1} \times 2^{e-b}}{2^{e-b}} = \varepsilon$$

ε est une borne supérieure de l'erreur relative.

Exercice 5. Suite de Muller-Kahane

On considère la suite numérique suivante :

$$\begin{aligned} u_0 &= 11/2 \\ u_1 &= 61/11 \end{aligned}$$

et

$$u_{n+1} = 111 - \frac{1130}{u_n} + \frac{3000}{u_n u_{n-1}}$$

1. Implantez cette suite dans Excel ou un tableur de votre choix, en Python et en C. Examinez les 30 premiers termes. Que concluez-vous ? Quelle est la limite de cette suite quand n tend vers $+\infty$?
2. Montrez que cette suite peut se définir comme une fonction de n avec

$$u_n = \frac{5^{n+1} + 6^{n+1}}{5^n + 6^n}$$

À l'aide de cette nouvelle expression, calculez la limite de la suite quand n tend vers $+\infty$. Que concluez-vous des implantations précédentes ?

3. Implantez la suite avec nouvelle expression. Comparez vos résultats.
Quel inconvénient présente cette approche ? Quelle parade peut-on proposer ?
4. On veut maintenant vérifier si une augmentation de la précision n'aurait pas résolu le problème. Implantez la suite en C avec la bibliothèque MPFR. Faites varier la précision. Qu'observez-vous ?
5. Pour finir, on essaie maintenant de nouvelles valeurs initiales :

$$\begin{aligned} u_0 &= 2 \\ u_1 &= -4 \end{aligned}$$

Refaites vos essais en variant la précision. Qu'observez-vous cette fois ? Peut-on conclure avec ces nouvelles valeurs initiales ?

Exercice 6. Comment calculer juste une intégrale ?

On donne la suite d'intégrales suivante :

$$I_n = \int_0^1 x^n e^{-x} dx$$

Problème : on veut calculer I_{50}

1. Montrer, par une intégration par partie, que

$$I_n = nI_{n-1} - \frac{1}{e}$$

2. Calculer I_0 , et donner un encadrement de I_n .
3. dans un script python, en partant de I_0 , calculer I_{50} en calculant les termes successifs de la suite à l'aide de la formule de la première question. On utilisera la bibliothèque `numpy`. Que pensez-vous du résultat ?
4. On propose de partir de l'estimation de I_{100} suivante :

$$I_{100} = \text{âge du capitaine} \times \frac{\sqrt{\pi}}{512 + \sin(3)} \approx 0.190347851$$

en supposant que le capitaine à 55 ans.

En écrivant I_{n-1} en fonction de I_n , calculer successivement $I_{99}, I_{98}, \dots, I_{50}$.

Le résultat vous semble-t-il plus convaincant ?

5. Faites varier l'âge du capitaine, entre 0 et 1000 ans, ou plus. Que constatez-vous ? Avez-vous une explication ?

(tiré de Jean-Michel Müller, cours d'arithmétique virgule flottante.)

Exercice 7. Double arrondi (inspiré du cours de Master de Guillaume Révy, LIRMM/Université de Perpignan)

Soit l'extrait de code suivant :

```
double a = 1848874847.0;
double b = 19954562207.0;

double c; c = a*b;
printf("c = %20.19e\n",c);
```

1. Compilez sans option et exécutez cet extrait de code.
2. Modifiez les types `double` en `float`. Que constatez-vous ?
3. Mêmes questions mais avec l'option de compilation `-mfpmath=387`. Consultez la documentation de `gcc`, et expliquez ce que fait cette option. Pouvez-vous expliquer les résultats obtenus avec cette option ?
4. Quel est le résultat exact du calcul de `c` ?
5. Écrivez les versions vectorielles AVX2, avec les types `__m256` et `__m256d`. et les instructions suivantes :

```
__m256 _mm256_mul_ps (__m256 a, __m256 b)
__m256d _mm256_mul_pd (__m256d a, __m256d b)
```

Qu'en déduisez vous sur les unités flottantes du jeu d'instruction vectoriel ?

Dans quel cas une transformation d'un code séquentiel en code vectoriel est-elle valide ?

1.3.3 Calculs sans erreur

Peut-on effectuer des calculs sans erreurs avec des nombres flottants ?

Oui sous certaines conditions. Ceci est rendu possible par quelques propriétés heureuses. Lemme de Sterbenz, algorithme de Dekker...

Lemme 1. Lemme de Sterbenz

Soient a et b de nombres représentés en virgule flottante positifs. Si

$$\frac{a}{2} \leq b \leq 2a$$

alors $a - b$ est exactement représenté en virgule flottante. $a - b$ est donc calculé sans erreur dans chacun des modes d'arrondis.

Exercice 8. Prouvez le lemme de Sterbenz, en se ramenant aux notations $x = M \times \beta^{e+1-p}$

On note $RN(x)$ l'arrondi au plus près de x (rappel).

Voici un lemme intéressant qui ouvre la voie à des calculs sans erreurs :

Lemme 2. Soient a et b deux nombres représentés en virgule flottante normalisés. On suppose que l'exposant de a est supérieur ou égal à celui de b . En l'absence de dépassement, si

$$s = RN(a + b)$$

et

$$r = (a + b) - s$$

alors, $|r| < |b|$ et r est aussi un nombre virgule flottant (représenté sans erreur)

Démonstration 1. 1. s est le nombre flottant le plus proche de $a + b$, donc, il est plus proche de $a + b$ que ne l'est a . On alors :

$$|(a + b) - s| \leq |(a + b) - a|$$

on a donc bien

$$|r| \leq |b|$$

2. notons $a = M_a \times \beta^{e_a+1-p}$ et $b = M_b \times \beta^{e_b+1-p}$, avec $|M_a|, |M_b| \leq \beta^p - 1$, et $e_a \geq e_b$ (hypothèse).

On sait donc que $a + b$ est un multiple de β^{e_b+1-p} , par voie de conséquence, s aussi, et finalement r aussi. On peut donc écrire

$$r = R \times \beta^{e_b-p+1}$$

Or, $|r| \leq |b| \implies |R| \leq |M_b| \leq \beta^p - 1$

$\implies r$ est exactement représentable en flottant.

Dans ce lemme, on voit donc la possibilité de calculer la somme de nombres flottants sans erreur, si on sait calculer s et r . C'est l'objet de l'algorithme *fast2sum*. Cet algorithme est présenté dans l'exercice suivant.

Exercice 9. Somme et transformation sans erreur

Chapitre 2

Complexité et performance

2.1 Introduction

2.2 Complexité asymptotique

2.2.1 Notations

En informatique, on utilise couramment les notations suivantes pour noter la complexité algorithmique. Elles se présentent de la façon suivante :

$$\begin{aligned}\mathcal{O}(g) &:= \{f \in \mathcal{F} \mid \exists c > 0, \exists N \in \mathbb{N} \text{ tels que } \forall n > N \text{ on a } |f(n)| \leq c \times |g(n)|\} \\ \Omega(g) &:= \{f \in \mathcal{F} \mid \exists c > 0, \exists N \in \mathbb{N} \text{ tels que } \forall n > N \text{ on a } c \times |g(n)| \leq |f(n)|\} \\ \Theta(g) &:= \{f \in \mathcal{F} \mid \exists a, b > 0, \exists N \in \mathbb{N} \text{ tels que } \forall n > N \text{ on a } a \times |g(n)| \leq |f(n)| \leq b \times |g(n)|\}\end{aligned}$$

Ces notations expriment le comportement asymptotique des fonctions g et f :

- On dira que f "est en $\mathcal{O}(g)$ " pour exprimer le fait que lorsque n tend vers l'infini, f est majorée par un multiple de la fonction g .
- La notation Ω indique un minorant de la fonction f quand n tend vers l'infini.
- Une fonction f en $\Theta(g)$ est encadrée par deux multiples de g , elle est à la fois en $\mathcal{O}(g)$ et en $\Omega(g)$. On dira encore qu'elle se comporte asymptotiquement comme g , ou encore qu'elle est du même ordre de grandeur.

En informatique, par abus de langage et pour simplifier, on confond souvent les notations $\mathcal{O}$, dite notation de Landau et Θ . En effet, une fonction en $\Theta(g)$ est une fonction qui est à la fois en $\mathcal{O}(g)$ et en $\Omega(g)$.

Pour des algorithmes, ce qui nous intéresse en informatique, on est parfois amené à distinguer les cas moyens et les "pires cas" (anglicisme peu heureux mais accepté par la communauté scientifique française).

2.2.2 Des algorithmes classiques et leur complexité

Un premier exemple pour illustrer cette notion de complexité, c'est le produit scalaire (sur des données de type `float`) :

```

r = 0.0f;
for(i=0; i<n; i++)
    r += u[i]*v[i];

```

On observe ce morceau de code. Il comporte une boucle `for`, qui compte n itérations. On compte les opérations effectuées par l'algorithme, en négligeant les lectures/écritures en mémoire :

- à l'intérieur de la boucle, on effectue une multiplication et une addition ;
- la boucle exécute n itérations. On compte donc n additions et n multiplications.

On conclut que cette opération à une complexité en $\mathcal{O}(n)$.

Un deuxième exemple, c'est le Produit vecteur-matrice (sur des données de type `float`) :

```

for(i=0; i<n; i++)
    r[i] = 0.0f;
    for(j=0; j<m; j++)
        r[i] += m[i][j]*v[j];

```

On observe ce morceau de code. Il comporte deux boucles `for`, dont l'une compte n itérations, et elle encapsule la deuxième boucle, qui, elle compte m itérations. On compte les opérations effectuées par l'algorithme, en négligeant les lectures/écritures en mémoire :

- à l'intérieur de la boucle intérieure, on effectue une multiplication et une addition ;
- la boucle intérieure comporte m itérations, cette boucle effectue donc : m additions et m multiplications ;
- la boucle extérieure exécute n affectations puis n boucles intérieures. On compte donc $m \times n$ additions et multiplications.

On conclut que cette opération à une complexité en $\mathcal{O}(m \times n)$. Si $m = n$, c'est à dire dans le cas d'une matrice carrée, on a n^2 additions/multiplications, et la complexité est en $\mathcal{O}(n^2)$

Exercice 10. On veut effectuer la somme de N nombres :

```

S=0;
for(i=0; i<N; i++)
    S=S+a[i];
return S;

```

1. À partir du programme précédent, et évaluer le nombre d'opérations significatives et sa complexité ;
2. Proposer différentes façons parallèle d'exécuter cet algorithme. Le temps de calcul est-il différent ? La complexité est-elle changée ;

Exercice 11. Produit de matrices (`uint64_t`) :

```

for(i=0; i<n; i++)
    for(j=0; j<n; j++){
        r[i][j] = 0UL;
        for(k=0; k<n; k++)
            r[i][j] += m1[i][k]*m2[k][j];
    }

```

1. Évaluer le nombre d'opérations et la complexité correspondante.
2. en remarquant que cet algorithme exécute n fois un produit matrice vecteur, que peut-on dire de sa complexité ? Est-ce cohérent avec vos résultats précédents.

2.3 Mesure de performances

2.3.1 Procédure

Les mesures de performances d'un exécutable utilisent un compteur de cycles d'horloge du processeur. Les performances elles-mêmes dépendent de nombreux paramètres, dont la plateforme d'exécution. Dans le cas des systèmes basés sur les processeurs de la famille **x86-64** (Intel®), le jeu d'instruction comporte **rdtsc** (*Read Time Stamp Counter*) et **rdtscp**, la version sérialisante. Cette instruction renvoie l'état du compteur d'horloge global du processeur. Cette instruction est accessible via le fichier d'en-tête **ccount.h** fourni, qui comporte plusieurs macros, dont la macro **STAMP()**, qui prend pour argument une variable de type **unsigned long long int**.

Tous les détails ont leur importance. Par exemple, les processeurs Intel® disposent de mécanismes d'amélioration de performances tels que le **turbo-boost**®. Ce dispositif permet à chaque cœur d'avoir sa propre horloge et la fréquence d'un cœur peut ainsi être accélérée et devenir plus rapide que celle du compteur d'horloge global accessible par **rdtsc**. Par conséquent, cette option doit normalement être désactivé pour les mesures de performances. Pour ce faire, il faut cependant disposer des droit **root** sur la plateforme.

On peut formuler la même remarque à propos de l'**hyper-threading**®, censé améliorer la gestion des **threads**, et qui facilitent le changement de contexte d'un **thread** à l'autre, avec le concept de cœurs virtuels. Ceci n'est important que dans le cas d'implantations algorithmes parallèles *multi-thread*.

De même, les options de compilation ont une grande importance sur les performances de l'exécutable obtenu.

Voici la procédure à suivre dans le code source :

1. il faut commencer par "chauffer" les caches, c'est à dire lancer l'exécution du code à tester quelques centaines de fois ;
2. il faut ensuite placer les macros **STAMP(START)** et **STAMP(STOP)** de part et d'autre du code à tester, le tout dans une boucle avec environ mille itérations ; il convient de calculer la différence **STOP-START** à chaque fois et évaluer le minimum (ou sauver chaque valeur dans un fichier).
3. il faut recommencer l'opération plusieurs fois avec des jeux de données différents (30 fois est un bon compromis) ;
4. le résultat attendu, c'est à dire le nombre de cycle estimé pour l'exécution du code à tester, c'est la moyenne des minimums obtenus pour chaque jeu de données.

Le calcul de la moyenne des minimums s'explique ainsi :

- le minimum pour chaque jeu de données correspond à l'exécution dans les conditions "idéales" de l'algorithme, c'est à dire sans perturbations dues au système d'exploitation (qui est pré-emptif dans la plupart des cas), avec l'état des mémoires caches optimal, avec le meilleur fonctionnement micro-architectural (prédiction de saut, pipeline, exécution *out-of-order*...)
- effectuer la moyenne de ces minimums permet d'éliminer l'influence des jeux de données : dans la plupart des algorithmes implantés qui ne sont pas en *temps constant*, le nombre d'instructions exécutées dépend des données en entrée.

— l'examen des résultats en détail permet de "pister" tous ces phénomènes.

Algorithme 1 présente un pseudo-code de cette procédure.

Algorithm 1 Pseudo-code de mesure de performances

Require: une fonction dont on veut mesurer les performances $f(x_1, \dots)$

Ensure: performance en nombre de cycles

```

 $(x_1, \dots) \xleftarrow{\$} \mathcal{X} \dots$ 
// chauffer les caches
for 1 = 1 to N_TEST do
     $out \leftarrow f(x_1, \dots)$ 
end for
// Mesurer les performances
#CC  $\leftarrow$  0
for 1 = 1 to N_SAMPLE do
     $(x_1, \dots) \xleftarrow{\$} \mathcal{X} \dots$ 
     $t \leftarrow MAX_t$ 
    for 1 = 1 to N_TEST do
        STAMP(START)
         $out \leftarrow f(x_1, \dots)$ 
        STAMP(STOP)
         $cc \leftarrow STOP - START$ 
        if  $cc < t$  then
             $t \leftarrow cc$ 
        end if
    end for
    #CC  $\leftarrow$  #CC + t
end for
return #CC/N_SAMPLE

```

Ceci peut s'accompagner d'expérimentations en variant les options de compilation et en examinant le code exécutable au besoin (en utilisant la commande `objdump -D %code_exec | less` dans un terminal).

Enfin, on peut aussi utiliser d'autres compteurs de performances. En particulier, on peut compter les instructions "retirées" (*retired instructions*) à l'aide du registre spécifique correspondant et l'instruction `rdpmc` (Read Performance Monitoring Counter). Ceci nécessite l'utilisation d'un module de noyau. L'utilité d'avoir le comptage des cycles d'horloge et des instructions, c'est d'évaluer le parallélisme d'instruction, l'*IPC*.

Une remarque que l'on peut faire, c'est que sur des machines partagées, ces compteurs de performances peuvent être accessibles à tous les utilisateurs. En conséquence, comme ces compteurs sont globaux, cela peut donner lieu à des attaques particulières d'un utilisateur malveillant vers d'autres utilisateurs, malgré la séparation des domaines...

2.4 Cas d'utilisation

2.4.1 Algorithmes récursifs

2.4.2 Suites définies par récurrence

1. Implanter une fonction dont le prototype est le suivant (implantation itérative) :
`unsigned long int factorial(int n)`, retournant $n!$.
2. Implanter une fonction récursive dont le prototype est le suivant :
`unsigned long int rec_factorial(int n)`, retournant $n!$.

On définit la suite de Fibonacci de la façon suivante :

$$\begin{aligned} u_1 &= 1, u_2 = 1, \\ u_n &= u_{n-1} + u_{n-2}, \quad \forall i > 2. \end{aligned}$$

3. Implanter une fonction dont le prototype est le suivant (implantation itérative) :
`unsigned long int fibonacci(int n)`, retournant u_n (le $n^{\text{ème}}$ terme de la suite de Fibonacci).
4. Implanter une fonction récursive dont le prototype est le suivant :
`unsigned long int rec_fibonacci(int n)`, retournant u_n .
5. Pour chacune de ces implantations, évaluer la complexité en fonction du paramètre d'entrée n . Que concluez-vous de ces complexités ?
6. Effectuez des mesures de performances (nombre de cycles)

Remarque 1. Vous pourrez vérifier que le rapport entre deux éléments consécutifs de la suite de Fibonacci (u_n et u_{n-1}) tend vers le nombre d'or, soit :

$$\varphi = \frac{1 + \sqrt{5}}{2}$$

2.4.3 Tableaux

Le but de ce travail est d'implanter et de tester différents algorithmes de traitement sur les tableaux.

L'implantation des algorithmes se fera en C dans Visual Studio (on pourra utiliser les fichiers sources du Moodle). Chacune des fonctions implantées sera testée depuis une fonction principale (`int main()`).

Allocations mémoire

1. Déclarer et allouer quatre tableaux de taille `SIZE` éléments de type `int` (voir le `#define`) :
 - un tableau `t0` en mémoire globale avec allocation statique ;
 - un tableau `t1` en mémoire locale (dans la fonction `int main()`) avec allocation statique ;
 - un tableau `t2` en mémoire locale (dans la fonction `int main()`) avec allocation dynamique (utilisez la fonction `malloc()`) ;
 - un tableau `t3` en mémoire locale (dans la fonction `int main()`) préfixé avec le mot clé `static` et avec allocation statique ;
2. Afficher les valeurs des pointeurs (format `%p`) et vérifiez avec le debugger de Visual Studio. Que constatez vous ?

Implantation

pour chaque fonction implantée, il vous faut la tester en l'appelant à partir de la fonction principale `main` et en affichant le résultat (en utilisant la première procédure implantée).

1. Planter une procédure d'affichage d'un tableau de `size` entiers dont le prototype est :
`void affiche_tableau(int *tab, int size);`
2. Planter une fonction d'initialisation d'un tableau de `size` entiers (déjà alloué en mémoire), affectant à chaque élément une valeur aléatoire entre 0 et `MAX_VAL-1`, macro définie en en-tête du fichier source (on utilisera la fonction `rand()` qui renvoie un entier entre 0 et 2^{31}) dont le prototype est :
`int init_tableau(int * tab, int size);`
3. Planter les fonctions de recherche dans le tableau précédent de l'élément minimum et l'élément maximum. Les prototypes sont les suivants :
 - `int min_tab(int * tab, int size);`
 - `int max_tab(int * tab, int size);`

Ces deux fonctions renvoient l'indice de l'élément correspondant dans le tableau.

4. Planter une procédure d'échange de deux éléments d'un tableau dont le prototype est le suivant (`elt1` et `elt2` sont les indices des éléments à échanger) :
`void swap(int * tab, int elt1, int elt2);`
5. Planter une procédure de tri sélection dont le prototype est le suivant :
`void selection_sort(int *tab, int size)`. Cette procédure modifie le tableau passé par son pointeur, et fera usage des fonctions et procédures précédentes.
6. Planter une fonction de tri rapide dont le prototype est le suivant :
`int quick_sort(int *tab, int size)`. Cette procédure modifie le tableau passé par son pointeur, et fera usage des fonctions et procédures précédentes. Elle sera bien sûr implantée de façon récursive.
7. Vérifier la complexité et comparer les performances (nombre de cycles) des deux procédures (en triant un grand tableau par exemple). Vous présenterez les résultats pour les deux fonctions de tri en variant la taille du tableau (10, 50, 100, 200, 400, 800...) et d'autres paramètres comme les options de compilation (`-O0`, `-O1`, `-O2` et `-O3`), type de données...
 Retrouvez-vous les résultats attendus en terme de performances ? Commentez.

Bibliographie

- [1] Bernard Goossens. *Architecture et micro-architecture des processeurs*. Springer, 2010.